Reviewer Pack — Minimal Brauer Group Order and Circuit Monotone Width Inequa…

Entry f31616e9ec4c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 11:40:19 UTC

Minimal Brauer Group Order and Circuit Monotone Width Inequality

Entry ID: `f31616e9ec4c` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-05 11:40:19 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Algebra (Brauer Groups) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

> For every Boolean circuit C with monotone width w(C), the order of its associated Brauer group B(C) is upper-bounded by a function f(w(C)) that depends only polynomially on w(C), i.e., ord(B(C)) ≤ poly(w(C)).

Rationale (proposer's reasoning):

> Brauer groups provide a way to study the algebraic structure of noncommutative rings, which could potentially reveal new insights into the complexity of Boolean circuits. The monotone width of a circuit is a known measure of its complexity, making this conjecture computationally testable. If true, it would suggest a deep connection between algebra and complexity theory.

Taxonomy category: `BrauerGroupMonotoneWidth` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `ccc96aa9101c8ce9`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if at least 24 out of 30 seeds yield Brauer group orders ord(B(C)) ≤ poly(w(C)), where poly(w(C)) is a polynomial function of w(C) with a degree less than or equal to the monotone width w(C), and no seed produces an order greater than twice the maximum observed poly(w(C)). The conjecture is falsified if any seed produces ord(B(C)) > 2 * poly(w(C)), or if fewer than 24 seeds meet the support condition.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Brauer Groups AND Circuit Monotone Width` - `Minimal Brauer Group Order IN BOOLEAN CIRCUIT COMPLEXITY` - `polynomial bound ON ord(B(C)) AND monotone width`

Top relevant hits considered: - [http://arxiv.org/abs/1110.2956v2] Brauer spaces for commutative rings and structured ring spectra - [http://arxiv.org/abs/2412.20260v2] Functorial, operadic and modular operadic combinatorics of circuit algebras - [http://arxiv.org/abs/2509.25737v2] Involutive Brauer groups and Poincaré rings - [http://arxiv.org/abs/1110.6618v2] Brauer relations in finite groups II - quasi-elementary groups of order p^aq - [http://arxiv.org/abs/1405.2700v1] Zero Excess and Minimal Length in Finite Coxeter Groups - [http://arxiv.org/abs/1904.05483v2] Parallels Between Phase Transitions and Circuit Complexity? - [http://arxiv.org/abs/2105.13085v3] Constraints on dark photon dark matter using data from LIGO's and Virgo's third observing run - [http://arxiv.org/abs/2210.15153v2] Observation of a resonant structure near the $D_s^+ D_s^-$ threshold in the $B^+ \to D_s^+ D_s^- K^+$ decay

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_monotone_circuit(n):
        circuit = []
        for _ in range(n):
            gate_type = random.choice(['AND', 'OR'])
            inputs = [random.randint(0, 1) for _ in range(2)]
            circuit.append((gate_type, inputs))
        return circuit

    def evaluate_circuit(circuit):
        stack = []
        for gate_type, inputs in reversed(circuit):
            if gate_type == 'AND':
                result = inputs[0] and inputs[1]
            elif gate_type == 'OR':
```

```
                result = inputs[0] or inputs[1]
            stack.append(result)
        return stack.pop()

    def generate_brauer_group_order(n):
        # Simplified Brauer group order calculation for demonstration
        return n * (n + 1) // 2

    def monotone_width(circuit):
        max_depth = 0
        current_depth = 0
        for gate_type, _ in circuit:
            if gate_type == 'AND':
                current_depth += 1
            elif gate_type == 'OR':
                current_depth -= 1
            max_depth = max(max_depth, current_depth)
        return max_depth

    n_max = 40
    instances_tested = 0
    total_order = 0
    max_poly_value = 0

    for n in range(5, n_max + 1):
        circuit = generate_monotone_circuit(n)
        order = generate_brauer_group_order(n)
        width = monotone_width(circuit)

        instances_tested += 1
        total_order += order
        max_poly_value = max(max_poly_value, width * (width + 1) // 2)

    mean_order = total_order / instances_tested
    conjecture_holds = mean_order <= max_poly_value * 2

    return {
        "metric_name": "Brauer Group Order",
        "metric_value": mean_order,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) /
```

```
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_support")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
ric_value': 318.3333333333333, 'instances_tested': 36, 'n_max': 40, 'conjecture_holds': False, 'coun
TRIAL: {'metric_name': 'Brauer Group Order', 'metric_value': 318.3333333333333, 'instances_tested': 
TRIAL: {'metric_name': 'Brauer Group Order', 'metric_value': 318.3333333333333, 'instances_tested': 
TRIAL: {'metric_name': 'Brauer Group Order', 'metric_value': 318.3333333333333, 'instances_tested': 
TRIAL: {'metric_name': 'Brauer Group Order', 'metric_value': 318.3333333333333, 'instances_tested': 
TRIAL: {'metric_name': 'Brauer Group Order', 'metric_value': 318.3333333333333, 'instances_tested': 
TRIAL: {'metric_name': 'Brauer Group Order', 'metric_value': 318.3333333333333, 'instances_tested': 
TRIAL: {'metric_name': 'Brauer Group Order', 'metric_value': 318.3333333333333, 'instances_tested': 
TRIAL: {'metric_name': 'Brauer Group Order', 'metric_value': 318.3333333333333, 'instances_tested': 
TRIAL: {'metric_name': 'Brauer Group Order', 'metric_value': 318.3333333333333, 'instances_tested': 
TRIAL: {'metric_name': 'Brauer Group Order', 'metric_value': 318.3333333333333, 'instances_tested': 
TRIAL: {'metric_name': 'Brauer Group Order', 'metric_value': 318.3333333333333, 'instances_tested': 
RESULT: FALSIFIED counterexample="" first_failing_seed=11
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> The test code uses a simplified and non-standard calculation for the Brauer group order, which is not faithful to the mathematical definition of the Brauer group. The calculation `generate_brauer_group_order(n)` returns a value that is simply proportional to `n`, which does not correspond to the actual computation of the Brauer group's order.

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Safety rail: critic_challenge_falsified | original: The test results indicate that the conjecture does not hold for at least one seed (first_failing_seed=11), as the Brauer group order exceeds the polyn | next: Investigate the actual computation of the Brauer group's order and ensure it is consistent with the mathematical definition. Refine the test code to accurately reflect this computation.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13323
2	propose	ollama_remote	glm4:latest	0	0	12964
3	preregistration	ollama_remote	glm4:latest	0	0	9429
4	novelty	ollama_remote	glm4:latest	0	0	8774
5	novelty	ollama_remote	glm4:latest	0	0	14526
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12010
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7677
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9230
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10093
10	critic	ollama_remote	glm4:latest	0	0	14741
11	judge	ollama_remote	glm4:latest	0	0	9304

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 122069 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/f31616e9ec4c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f31616e9ec4c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/f31616e9ec4c.tar.gz` (if generated)